Documento técnico del proyecto

Proyecto: SalesHealth Analytics
Asignatura: Gestión de Datos - Data Management
Universidad: Universidad Alfonso X el Sabio
Autor: Rubén Elices Rodríguez
Curso: 3º Ingeniería Matemática

1. Introducción

El objetivo de este proyecto ha sido construir un entorno analítico completo a partir de datos operacionales de una empresa ficticia dedicada a la venta de productos de salud. La empresa dispone de información dispersa en distintas áreas de negocio: clientes, ventas, productos, ofertas, devoluciones, tiendas, inventario y almacenes. Esta estructura es adecuada para registrar operaciones del día a día, pero no es la más cómoda para analizar el comportamiento del negocio o calcular métricas avanzadas de cliente.

Por ese motivo, el trabajo se ha planteado como un flujo completo de datos:

```
JSON fuente -> PostgreSQL OLTP -> ETL -> Data Warehouse -> CLTV/RFM/Clustering -> Dashboard
```

El resultado final es un Data Warehouse en PostgreSQL, un conjunto de notebooks analíticos y un dashboard interactivo en Streamlit. Sobre este entorno se calculan métricas de cliente como CLTV, churn proxy, segmentación RFM y clustering mediante PCA + K-Means.

El dataset contiene 17 ficheros JSON que representan una base de datos operacional con 5.750 clientes, 20 tiendas en Madrid, 50 productos y 20.000 ventas realizadas entre 2020 y 2025. La tabla de líneas de venta contiene 42.555 registros y los ingresos totales consolidados ascienden a 9.678.678,67 euros.

2. Arquitectura general

La arquitectura se ha diseñado separando claramente tres niveles:

1. **Datos origen:** ficheros JSON originales y base OLTP restaurada en PostgreSQL.
2. **Capa analítica:** Data Warehouse en esquema `dwh`, con modelo dimensional.
3. **Explotación:** notebooks de análisis, CSVs finales y dashboard Streamlit.

Esta separación es importante porque cada capa tiene una finalidad distinta. La base OLTP está pensada para registrar operaciones: una venta, una línea de venta, una devolución o un movimiento de inventario. En cambio, el DWH está pensado para consultar y agregar información: ingresos por cliente, margen por producto, ventas por fecha, devoluciones por motivo o segmentación de clientes.

La base OLTP se encuentra en PostgreSQL en el esquema `public`. El Data Warehouse se construye en el esquema `dwh`. El proceso ETL, desarrollado en Python con pandas y SQLAlchemy, extrae los datos del OLTP, aplica transformaciones e imputaciones, valida la consistencia y carga las tablas finales del DWH.

3. Modelo Entidad-Relación OLTP

El primer paso fue entender la estructura transaccional original. Para ello se documentaron las 17 tablas del sistema OLTP en formato DBML y se generó el diagrama Entidad-Relación.

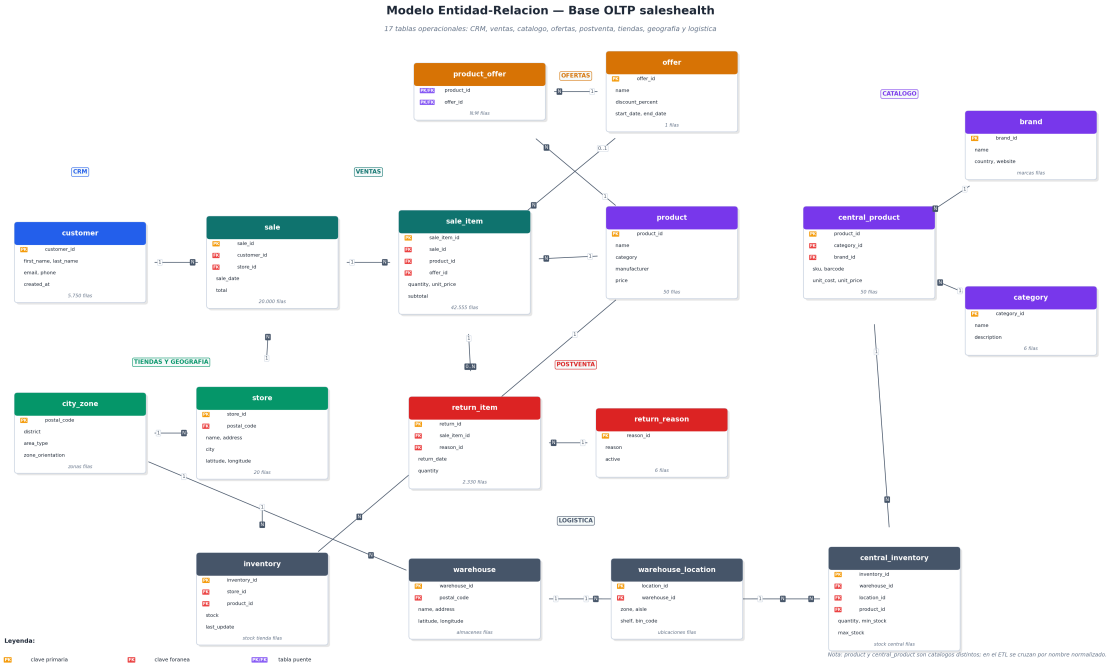

Diagrama ER

El modelo OLTP está organizado por áreas funcionales:

Área	Tablas principales	Función
CRM	`customer`	Información de clientes
Ventas	`sale`, `sale_item`	Cabeceras y líneas de venta
Catálogo	`product`, `central_product`, `brand`, `category`	Productos, marcas, categorías y costes
Ofertas	`offer`, `product_offer`	Promociones y relación producto-oferta
Postventa	`return_item`, `return_reason`	Devoluciones y motivos
Tiendas y geografía	`store`, `city_zone`	Localización de tiendas y zonas de Madrid
Logística	`inventory`, `warehouse`, `warehouse_location`, `central_inventory`	Stock en tienda y almacenes

La relación central del modelo es:

```
customer -> sale -> sale_item -> product
```

Cada cliente puede realizar varias ventas, cada venta puede contener varias líneas, y cada línea corresponde a un producto vendido. A partir de esta estructura se puede calcular la facturación histórica

de cada cliente.

También se detectó una relación relevante entre `sale_item` y `return_item`, que permite identificar si una línea de venta fue devuelta. Esta información se incorporó posteriormente al DWH mediante el flag `has_return` y la dimensión `dim_return_reason`.

Un punto importante del análisis fue que `product` y `central_product` no son exactamente la misma tabla. Aunque ambas contienen productos, sus identificadores no se corresponden directamente. La tabla `product` representa el catálogo de tienda, mientras que `central_product` contiene información centralizada como el coste unitario. Para poder calcular márgenes fue necesario cruzarlas por nombre normalizado, no por `product_id`.

---

4. Análisis exploratorio y calidad de datos

Antes de construir el Data Warehouse se realizó un EDA en `01_eda.ipynb`. El objetivo de esta fase fue conocer los datos, detectar problemas de calidad y decidir cómo tratarlos. Esta fase es fundamental porque las decisiones tomadas aquí afectan directamente al ETL, al CLTV y al clustering.

Los principales hallazgos fueron:

Problema detectado	Impacto	Decisión tomada
Clientes con email, nombre o teléfono nulos	Podrían provocar pérdida de clientes o errores en análisis posteriores	Imputar valores por defecto sin eliminar clientes
Aproximadamente 5.000 clientes con `created_at` futuro	Si se usa esa fecha, la antigüedad del cliente puede ser cero o negativa	Usar fechas de compra para calcular antigüedad analítica
`offer_id` nulo en prácticamente todas las líneas de venta	La FK a ofertas no puede quedar nula en el DWH	Crear fila centinela `offer_id = -1` como “Sin oferta”
Un producto sin coste conocido	No se podría calcular margen correctamente	Imputar coste con la mediana
Categoría `Domótica Salud` no presente en tabla `category`	Inconsistencia de catálogo	Reclasificar como `Diagnóstico`
Una venta con discrepancia de 3 euros entre cabecera y líneas	Posible descuadre entre `sale.total` y suma de `sale_item`	Usar la suma de líneas y marcar `total_corrected = true`

La regla principal del proyecto fue no eliminar clientes. Por tanto, cuando aparecieron valores nulos o inconsistentes, se imputaron o documentaron. Esta decisión mantiene el universo completo de 5.750 clientes y evita sesgos en métricas posteriores.

---

5. Diseño del Data Warehouse

Una vez entendido el modelo OLTP, se diseñó un modelo dimensional en estrella. Este tipo de modelo es más adecuado para análisis porque reduce la complejidad de las consultas y organiza la información alrededor de una tabla de hechos.

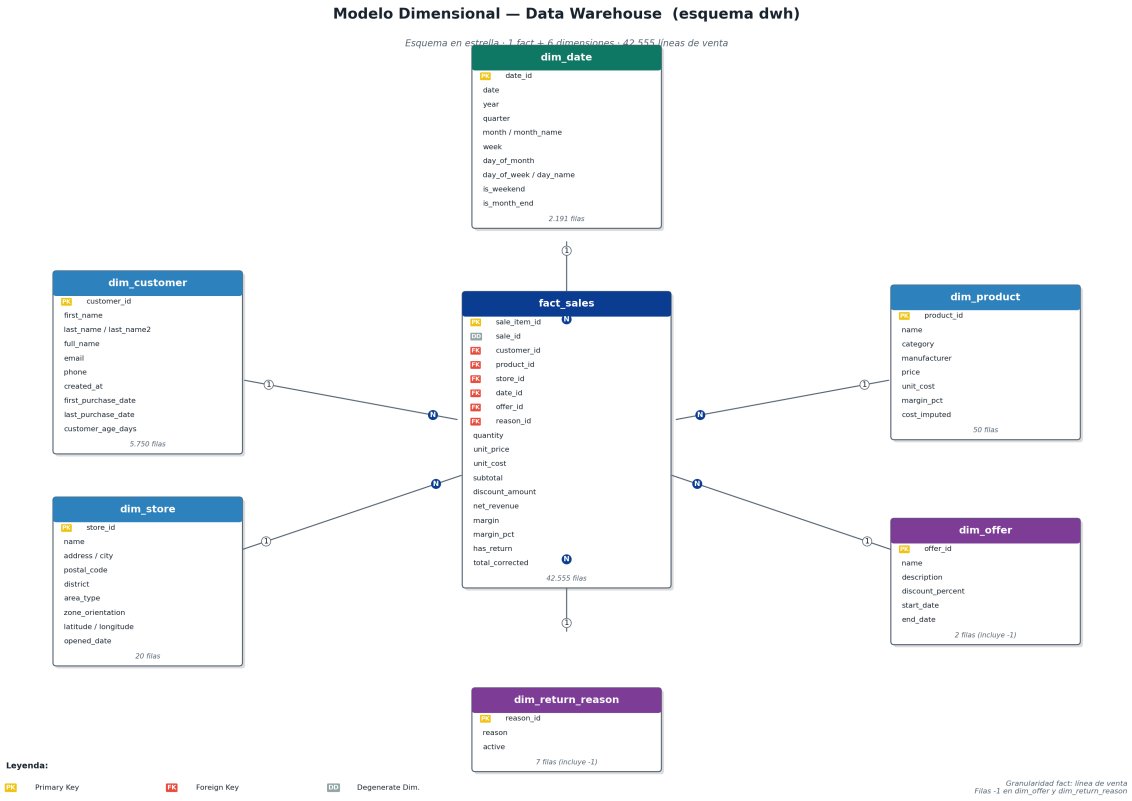

Diagrama dimensional

La tabla central del DWH es `fact_sales`. Su granularidad es una fila por línea de venta, es decir, una fila por registro de `sale_item`. Esta decisión permite analizar las ventas con mucho detalle: por cliente, producto, tienda, fecha, oferta y motivo de devolución.

El DWH está formado por una tabla de hechos y seis dimensiones:

Tabla	Tipo	Filas esperadas	Descripción
`fact_sales`	Hechos	42.555	Líneas de venta enriquecidas con métricas
`dim_date`	Dimensión	2.191	Calendario analítico 2020-2025
`dim_customer`	Dimensión	5.750	Clientes con información imputada y fechas de compra
`dim_product`	Dimensión	50	Productos con coste, categoría y margen
`dim_store`	Dimensión	20	Tiendas enriquecidas con zona geográfica
`dim_offer`	Dimensión	2	Ofertas, incluyendo fila `−1`
`dim_return_reason`	Dimensión	7	Motivos de devolución, incluyendo fila `−1`

En `fact_sales` se calcularon métricas que no estaban directamente disponibles en el OLTP:

Métrica	Definición
`discount_amount`	Descuento aplicado a la línea
`net_revenue`	Ingreso neto después de descuento
`margin`	Ingreso neto menos coste de los productos vendidos
`margin_pct`	Margen relativo sobre ingreso neto
`has_return`	Indica si la línea fue devuelta
`total_corrected`	Indica si la venta pertenece al descuadre detectado

La dimensión `dim_offer` y la dimensión `dim_return_reason` incluyen filas especiales con identificador `-1`. Estas filas representan "Sin oferta" y "Sin devolución". Gracias a esto, la tabla de hechos no necesita dejar claves foráneas nulas y conserva la integridad referencial.

---

6. Proceso ETL

El ETL se implementó en `03_etl.ipynb`. Su función es mover los datos desde el OLTP hasta el DWH aplicando las decisiones de limpieza definidas en el EDA.

El proceso sigue tres bloques:

```
Extracción -> Transformación -> Carga y validación
```

En la extracción se leen las tablas necesarias del esquema `public`: clientes, ventas, líneas de venta, productos, catálogo central, tiendas, zonas, ofertas, devoluciones y motivos de devolución.

En la transformación se construyen las dimensiones y la tabla de hechos:

Transformación	Explicación
`dim_date`	Se genera un calendario completo para el periodo 2020-2025
`dim_customer`	Se imputan nulos y se calculan primera compra, última compra y antigüedad
`dim_product`	Se cruza producto con catálogo central y se imputa el coste faltante
`dim_store`	Se añade información geográfica desde `city_zone`
`dim_offer`	Se cargan ofertas y se añade la fila `-1`
`dim_return_reason`	Se cargan motivos y se añade la fila `-1`
`fact_sales`	Se calculan ingresos, costes, márgenes, devoluciones y flags de calidad

La carga se diseñó para ser idempotente. Esto significa que ejecutar el notebook varias veces debe producir siempre el mismo resultado, sin duplicar filas. Para conseguirlo, antes de insertar datos se hace

`TRUNCATE` de las tablas del DWH y después se vuelven a cargar en el orden correcto.

El orden es importante por las claves foráneas. Primero se cargan las dimensiones y al final la tabla de hechos. Además, las filas `-1` de `dim_offer` y `dim_return_reason` se reinsertan dentro del propio ETL para no perderlas al truncar.

Al final del proceso se validan los conteos y los ingresos. El total de ingresos del DWH coincide con el total esperado:

```
Ingresos totales DWH = 9.678.678,67 euros
```

---

7. Cálculo de CLTV y métricas de cliente

El cálculo de métricas de cliente se desarrolló en `04_cltv.ipynb`. Esta fase parte del DWH y agrega la información a nivel de cliente.

Para cada cliente se calcularon:

Variable	Significado
`ingresos_t`	Ingresos netos históricos del cliente
`margen_total`	Margen total acumulado
`margen_t`	Ratio de margen sobre ingresos
`num_ventas`	Número de ventas realizadas
`frecuencia_t`	Ventas por mes activo
`r_t`	Antigüedad del cliente en meses
`ticket_medio`	Importe medio por venta
`dias_sin_compra`	Días desde la última compra
`churn_proxy`	Indicador de posible abandono
`tasa_devolucion`	Proporción de líneas devueltas

En la versión actual del proyecto, el CLTV exportado sigue la fórmula indicada en el enunciado:

```
CLTV = Ingresos_t × Margen_t × Frecuencia_t × R_t
```

Esta definición combina cuatro ideas: el volumen económico del cliente (`ingresos_t`), la rentabilidad relativa (`margen_t`), la frecuencia de compra (`frecuencia_t`) y la duración de la relación (`R_t`). Además, se conserva `margen_total` como métrica auxiliar para interpretar la rentabilidad histórica realizada.

Los resultados principales son:

Indicador	Valor
Clientes analizados	5.750
Ingresos totales	9.678.678,67 €
CLTV medio	12.712,47 €
CLTV mediano	59,96 €
Segmento Alto	1.386 clientes
Segmento Medio	2.931 clientes
Segmento Bajo	1.433 clientes

La segmentación por CLTV se hizo mediante percentiles:

Segmento	Criterio	Interpretación
Alto	Por encima de P75	Clientes de mayor valor económico
Medio	Entre P25 y P75	Clientes con valor intermedio
Bajo	Por debajo de P25	Clientes con bajo valor histórico

El segmento Alto concentra la mayor parte de los ingresos del negocio, lo que confirma que la cartera de clientes está muy desbalanceada: pocos clientes generan una proporción muy elevada de la facturación.

---

8. Segmentación RFM y churn proxy

Además del CLTV, se construyó una segmentación RFM. Esta técnica clasifica clientes en función de tres dimensiones:

Dimensión	Pregunta que responde
Recency	¿Hace cuánto compró por última vez?
Frequency	¿Con qué frecuencia compra?
Monetary	¿Cuánto dinero genera?

La segmentación RFM es útil porque traduce métricas numéricas en perfiles de negocio más accionables. Por ejemplo, permite distinguir clientes valiosos pero inactivos, clientes frecuentes pero de bajo importe o clientes recientes con potencial de crecimiento.

También se calculó un `churn_proxy`, entendido como una aproximación al riesgo de abandono. No se trata de un modelo predictivo supervisado, porque no existe una etiqueta real de abandono, sino de una regla de negocio basada en los días transcurridos desde la última compra.

Estas métricas complementan el CLTV porque permiten interpretar no solo cuánto valor ha generado un cliente, sino también su estado actual.

---

9. PCA y clustering

La fase de segmentación automática se desarrolló en `05_pca_clustering.ipynb`. El objetivo fue agrupar clientes según su comportamiento utilizando técnicas de Machine Learning.

El proceso aplicado fue:

```
Selección de variables -> Tratamiento de outliers -> Normalización -> PCA -> K-Means ->
Interpretación
```

Las variables utilizadas fueron:

Variable	Motivo
`ingresos_t`	Captura volumen económico
`margen_t`	Captura rentabilidad relativa
`frecuencia_t`	Captura hábito de compra
`ticket_medio`	Captura tamaño medio de compra
`tasa_devolucion`	Captura comportamiento postventa

Antes de aplicar K-Means se normalizaron las variables con `StandardScaler`, ya que tienen escalas muy distintas. Sin normalización, las variables monetarias dominarían el clustering.

PCA se utilizó principalmente para visualizar los clientes en dos dimensiones y entender la estructura de los datos. El clustering final se aplicó sobre el espacio normalizado de variables, no solo sobre las dos componentes PCA.

El resultado final identifica tres clusters:

Cluster	Clientes	Interpretación
VIP	750	Clientes con mayor volumen económico y frecuencia
Regular	4.560	Mayoría de clientes, bajo volumen individual
Bajo valor	440	Clientes de bajo valor y alta tasa de devolución

La comparación entre segmentación por CLTV y K-Means muestra que ambas no son exactamente iguales. Esto es positivo desde el punto de vista analítico, porque el clustering no solo replica los percentiles de CLTV, sino que aporta otra lectura basada en comportamiento multivariable.

---

10. Dashboard interactivo

Como capa final de explotación se desarrolló un dashboard en Streamlit. El archivo principal es `app.py` y las páginas están organizadas dentro de la carpeta `pages/`.

El dashboard permite consultar:

Página	Contenido
Inicio	KPIs generales y arquitectura del proyecto
KPIs	Resumen ejecutivo de clientes, ingresos y segmentos
CLTV	Distribución, filtros, top clientes y segmentos
RFM	Segmentación por recencia, frecuencia y monetario
Clustering	Visualización PCA, perfiles y distribución de clusters
Cliente	Búsqueda individual y comparación con clientes similares

Esta parte no sustituye al análisis técnico de los notebooks, sino que lo convierte en una herramienta exploratoria. Permite navegar los resultados sin ejecutar código y facilita una lectura de negocio.

---

11. Conclusiones

El proyecto cumple el objetivo principal de construir un entorno analítico completo sobre datos operacionales. Se ha partido de ficheros JSON y una base OLTP normalizada, se ha diseñado un Data Warehouse dimensional y se han desarrollado análisis avanzados de cliente.

Las principales decisiones técnicas fueron:

- Mantener todos los clientes, imputando valores cuando era necesario.
- Usar un modelo estrella con granularidad de línea de venta.
- Crear filas centinela para evitar claves nulas en oferta y devolución.
- Calcular métricas económicas en la tabla de hechos para simplificar análisis posteriores.
- Separar la lógica de preparación de datos, análisis y visualización.

Desde el punto de vista de negocio, el análisis muestra una cartera de clientes muy concentrada: un grupo reducido de clientes de alto valor explica la mayor parte de los ingresos. Esto justifica estrategias diferenciadas de fidelización para clientes VIP, recuperación para clientes en riesgo y control de devoluciones para segmentos menos rentables.

Como mejoras futuras se podrían incorporar modelos predictivos de churn, una definición probabilística de CLTV futuro y actualización automática del DWH mediante jobs programados. También sería conveniente confirmar con la profesora la definición exacta de CLTV que se desea usar en la entrega final, ya que existen varias formulaciones válidas según el objetivo analítico.

---

12. Archivos entregables

Entregable	Archivo
Enunciado	`GD_ProyectoFinal.pdf`
Modelo ER OLTP	`saleshealth_schema.dbml`, `diagrama_er.png`
Modelo dimensional DWH	`saleshealth_dwh_schema.dbml`, `diagrama_dimensional.png`
DDL del DWH	`02_ddl_dwh.sql`
EDA	`01_eda.ipynb`
ETL	`03_etl.ipynb`
CLTV y métricas de cliente	`04_cltv.ipynb`, `cltv_resultados.csv`
PCA y clustering	`05_pca_clustering.ipynb`, `clustering_resultados.csv`
Dashboard	`app.py`, `pages/`, `utils/`
Documento técnico	`06_documento_tecnico.md`